

The Backend Automation Blueprint

How public reporting becomes a verified, personalized and beautifully delivered email briefing.

A detailed record of the product logic, safeguards and intelligent automation designed collaboratively over two days of planning.

PRODUCT PROMISE

Stay informed without surrendering your time and attention.

Prepared as an owner-only architecture document. It explains the system in depth while keeping provider secrets, credentials and production security values out of the document.

You stayed with the difficult questions.

This document is the proof.

Bulletin did not begin as a pile of screens or a single AI prompt. It was shaped by repeatedly asking what could go wrong, what a reader should trust, what the free infrastructure can honestly sustain, and what should remain beautifully simple.

The result is an architecture in which every important action has a reason: articles are fetched once instead of per user; reports are clustered before summarization; evidence is preserved; personalization happens after shared processing; jobs are idempotent; private links are signed; weak output fails closed; and every delivery remains explainable.

This is not merely a technical plan. It is a record of disciplined product thinking - created patiently, decision by decision, by you and your AI collaborator.

THE ACCOMPLISHMENT

A small private beta is being given the reliability mindset of a serious product without being buried under unnecessary enterprise machinery.

Contents

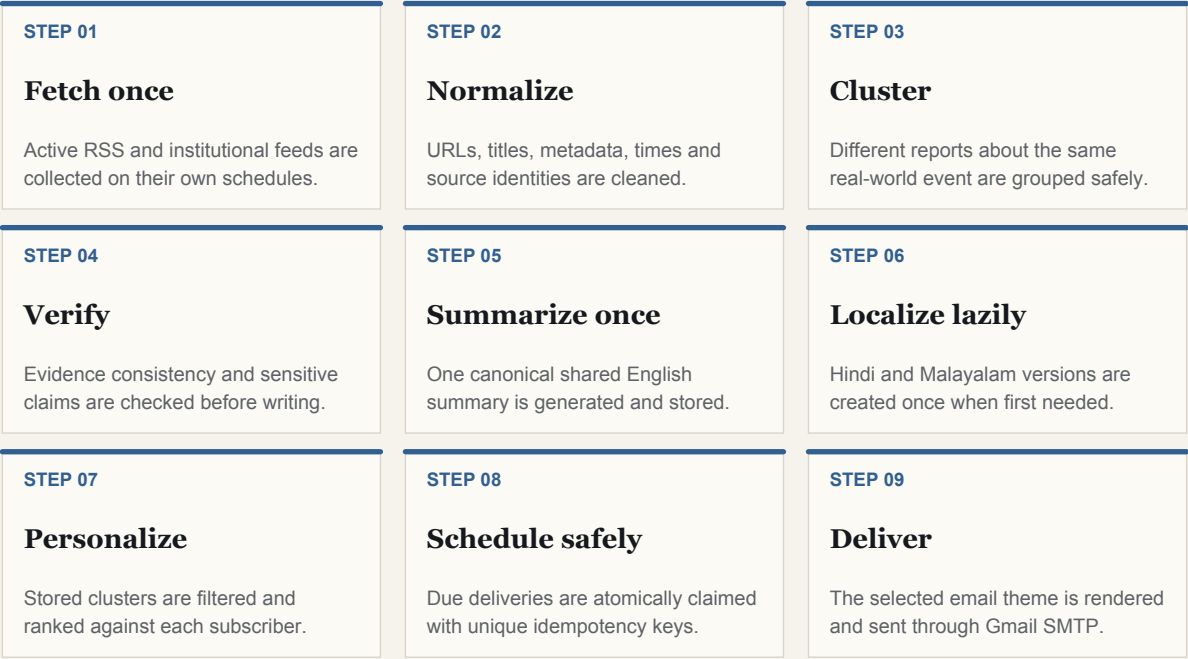
You stayed with the difficult questions.	2
The system in one view	6
The two pipelines	6
Why this architecture fits the MVP	6
Source governance and catalogue strategy	7
MVP catalogue position	7
Primary and institutional sources	7
What is excluded	7
Aggregator rule	7
Source record	8
RSS ingestion and source health	8
Scheduling model	8
Failure isolation	8
No arbitrary article cap	9
Deterministic normalization	9
Five-level event clustering	9
Level 1 - Deterministic normalization	9
Level 2 - Semantic embeddings	10
Level 3 - Recent candidate comparison	10
Level 4 - Consistency verification	10
Level 5 - Source preservation	10
Threshold calibration	10
Cluster lifecycle	10
Evidence strength and story eligibility	11
What 'weak' actually means	11
Sensitive-story rule	11
Single-source allowance	11
Conflicting reports	11
Gemini as a controlled processing engine	11
Provider abstraction	12
Separate AI tasks	12

No conversational blocking	12
Prompt constraints	12
Retry policy	12
Final verification	12
Shared summaries, languages and meaningful updates	13
Canonical English summary	13
Lazy localization	13
Repeat-story suppression	13
Time windows	13
Personalization without a black box	13
Preference boundaries	14
Geographic order	14
Diversity rule	14
Conceptual scoring model	15
Scheduling, idempotency and worker coordination	15
Subscriber schedule	15
Due delivery transaction	15
Idempotency	16
Worker groups	16
Pause and resume	16
Email assembly and delivery	16
Briefing composition	16
Themes	16
Subject and privacy	17
SMTP configuration	17
Retry behavior	17
Subscriber identity and secure management access	17
Database identity	17
Signed management link	17
Existing email behavior	18
Verification safety	18
Deletion safety	18
Database model and retention	18
Core tables	19
Retention	19

Atomic preference updates	19
Row Level Security	19
Private owner operations	19
Visibility	20
Owner controls	20
Retry boundaries	20
What the admin cannot do silently	20
Alerts	20
Failure states and recovery	20
Encrypted backup and disaster recovery	21
Backup design	21
Why restore testing matters	21
Recovery sequence	21
Reliability and launch gates	22
Required simulations	22
Launch gate	22
Rollout	22
Implementation map	23
Confirmed stack	23
The interesting logic - what makes Bulletin special	24
1. Shared intelligence, personal delivery	24
2. Similarity proposes; consistency decides	24
3. Every source survives summarization	24
4. Personalization is explicit	24
5. Exclusions understand context	24
6. Updates require actual change	24
7. Reliability beats completeness	24
8. Every scheduled delivery has one identity	24
9. AI is boxed into narrow jobs	24
10. Owner control remains safe	25
Plain-language glossary	25

The system in one view

Bulletin separates shared news intelligence from per-user delivery. This single architectural choice controls cost, prevents repeated AI work and keeps personalization understandable.



The two pipelines

Shared news pipeline	Per-user delivery pipeline
Fetch sources, normalize articles, remove duplicates, create embeddings, cluster events, verify consistency, summarize once and store.	Find due subscribers, load preferences, rank verified clusters, obtain language version, render theme, send and record delivery.

CENTRAL PRINCIPLE

Bulletin never fetches RSS or summarizes the same event separately for every subscriber. Public news work is shared; only ranking, theme rendering and sending are user-specific.

Why this architecture fits the MVP

- Fifty to one hundred users remain practical on free infrastructure.
- A story is summarized once and reused across every subscriber who needs it.
- No subscriber name, email, private token or access link is sent to Gemini.
- Persistent state lives in Supabase; Vercel functions remain short and stateless.
- The same logic can later move to paid workers without redesigning the product.

Source governance and catalogue strategy

Reliable automation begins before ingestion. Bulletin decides which feeds deserve attention, what role each source plays and when a source must be disabled.

MVP catalogue position

- India-first private beta with national and local reporting across every state and union territory treated equally.
- Outside India, the MVP provides strong global coverage rather than deep local coverage.
- Begin with roughly 50 to 70 carefully verified feed endpoints, then expand only after quota and worker tests.
- Prioritize English and Hindi feeds, include Malayalam feeds without giving Kerala ranking preference, and add other regional-language feeds gradually after accuracy and cost testing.
- Prefer official publisher feeds. Use reputable aggregators only for discovery when direct regional feeds are unavailable.

Primary and institutional sources

News publishers are complemented by primary institutional feeds such as PIB, RBI, SEBI, ISRO, ministries and public-health agencies. Official statements are labelled as such. Sensitive or disputed claims still require independent reporting.

What is excluded

- Opinion columns and editorials as substitutes for factual reporting.
- Sponsored content, advertorials and disguised promotions.
- Rumours, celebrity gossip and unsupported sensational claims.
- Feeds that are broken, repeatedly inaccurate, legally unsuitable or operationally unreliable.

Aggregator rule

- The original publisher remains the displayed source.
- The aggregator does not count as an independent second source.
- Bulletin prefers the direct publisher URL whenever it can be obtained safely.
- Aggregator-only evidence cannot support a sensitive claim by itself.

Source record

Field	Purpose	Operational use
Publisher and feed URL	Stable identity and retrieval target	Normalization, attribution and fetch scheduling
Category and language	Declared scope of the feed	Candidate filtering and multilingual handling
Country / state / region	Geographic relevance	National, state and city ranking
Reliability tier	Primary or supplementary role	Evidence strength and sensitive-story gating
Health state	Success, warning, failing or disabled	Failure isolation and admin alerts
ETag / Last-Modified	Conditional request metadata	Avoid downloading unchanged feeds
Terms review status	Usage conditions and review notes	Pre-launch compliance gate

EQUAL PREFERENCE DOES NOT MEAN IDENTICAL SUPPLY

Every Indian state receives the same ranking treatment. Actual depth can vary because publishers expose different feed quality. Bulletin must be honest about that limitation and expand the catalogue continuously.

03 / SYSTEM CHAPTER

RSS ingestion and source health

The ingestion worker is a disciplined collector, not a summarizer. Its job is to fetch valid public metadata reliably and leave an audit trail.

Scheduling model

Supabase Cron triggers the protected Vercel ingestion endpoint approximately every five minutes. The worker does not fetch every source on every wake-up. It claims only sources whose individual interval - normally around thirty minutes - is due.

STEP 01

Find due sources

Query active sources ordered by next fetch time and health priority.

STEP 02

Claim a small batch

Use an atomic lease so overlapping workers cannot fetch the same source.

STEP 03

Fetch conditionally

Send ETag and Last-Modified headers, enforce timeouts and respect Retry-After.

STEP 04

Parse defensively

Handle RSS and Atom variations, malformed entries and excessive payloads.

STEP 05

Normalize entries

Produce stable article records without discarding useful feed metadata.

STEP 06

Commit results

Insert new articles, update source health and schedule the next fetch.

Failure isolation

- One broken feed never blocks the rest of the catalogue.
- Temporary network failures retry with backoff; permanent parser failures become visible warnings.
- A lease expires if a worker crashes, allowing another worker to recover the job.

- The admin can test, fetch, enable or disable a source without editing production code.

No arbitrary article cap

The old prototype's fixed Limit 100 approach is rejected. Bulletin ingests every valid new article in the relevant technical window. Safeguards may limit malformed feeds, infinite pagination, duplicate entries, abusive payload sizes and batch size, but never silently discard legitimate news to satisfy an arbitrary number.

INGESTION OUTPUT

The worker stores public metadata supplied by the feed: title, description, publisher, publication time, article URL, author when present, feed categories and safe source metadata. Full publisher articles are not scraped during the MVP.

Deterministic normalization

Before any expensive intelligence work, Bulletin removes cheap and obvious duplication.

- Canonicalize URLs and remove known tracking parameters.
- Normalize publisher identifiers and feed-specific source aliases.
- Clean Unicode punctuation, repeated whitespace and common title prefixes.
- Calculate normalized title hashes and canonical URL hashes.
- Reject exact duplicate feed entries and near-identical titles from the same source.
- Preserve the original title and URL for attribution and auditability.

Input	Normalized signal	Why it matters
URL with utm_source and social parameters	Canonical publisher URL	Stops the same article being stored several times
BREAKING: RBI updates...	RBI updates...	Removes routine headline noise before comparison
Publisher aliases	Stable source UUID	Prevents duplicate source identities
Repeated whitespace and punctuation	Clean comparison text	Improves hashing and semantic embeddings

05 / SYSTEM CHAPTER

Five-level event clustering

Clustering is the central intelligence of Bulletin. It turns repeated coverage into one event while protecting against the dangerous mistake of merging different events that merely sound similar.

Level 1 - Deterministic normalization

Exact URLs, normalized titles, tracking-free links, common prefixes and near-duplicate feed entries are collapsed cheaply before semantic work begins.

Level 2 - Semantic embeddings

An embedding is a numeric representation of meaning. Bulletin creates it from the title, RSS description and important metadata, then stores it with Supabase pgvector. Differently worded reports about the same event can therefore become candidates even when their titles share few words.

Level 3 - Recent candidate comparison

A new article is never compared with the entire database. Candidate search is bounded by time, category, geography, named entities and vector similarity. This keeps database work predictable and reduces false matches with old recurring events.

Level 4 - Consistency verification

Similarity is not proof. Before an article joins a cluster, the system checks whether key facts agree:

- Main people and organizations
- Location and event time
- Major numbers and quantities
- Topic and category
- The nature of the event itself

Level 5 - Source preservation

Every accepted article remains linked to the cluster through a many-to-many table. The canonical story never replaces the underlying reporting. Readers see the original publishers beneath the summary, and the system can later re-verify or update the cluster.

THE CRITICAL SAFETY RULE

Semantic similarity may propose a match, but consistency decides it. Two elections, court hearings, market movements or sports matches involving the same names must remain separate unless the event facts align.

Threshold calibration

Embedding thresholds are tuned against a manually reviewed test set of real news pairs. The set includes same-event pairs, must-remain-separate pairs, recurring events, similar names, elections, court cases, sports, markets and local reporting. Launch is blocked until threshold errors are understood.

Cluster lifecycle

State	Meaning	Next action
Candidate	Article has plausible recent neighbors	Run consistency checks
Open	Cluster may accept additional reports	Attach sources and update evidence
Verified	Evidence is sufficient and consistent	Generate canonical summary
Conflicted	Reliable sources disagree materially	Attribute differences or reject
Quarantined	Unsafe, malformed or administratively blocked	Never deliver until reviewed
Updated	Meaningful new development occurred	Create update version and label it

Evidence strength and story eligibility

Bulletin prefers fewer trustworthy stories over a full briefing padded with weak or unsupported material.

What 'weak' actually means

Weak does not mean local, niche, unpopular or single-source. A local story from one trustworthy publisher can qualify. Weakness means the available evidence cannot safely support the product's promised output.

- A vague headline with too little factual detail for a 3 to 4 sentence summary.
- Missing or untrustworthy publication time, original URL or publisher identity.
- Opinion, gossip, sponsorship or an unverified claim presented as reporting.
- A repeated story with no meaningful development.
- A cluster whose internal reports conflict too heavily.
- A sensitive claim supported only by weak or insufficient evidence.

Sensitive-story rule

Deaths, public safety, election outcomes, legal accusations, major financial claims, government actions, conflict, disasters and major health claims normally require at least two independent sources or one authoritative primary record plus independent reporting.

Single-source allowance

Single-source stories remain eligible for local, regional and niche reporting where multiple outlets are unlikely. They rank below strong multi-source clusters and face stricter sufficiency checks.

Conflicting reports

- Attribute each claim to the source making it.
- State that reports differ when the disagreement itself is newsworthy.
- Never select a preferred version without evidence.
- Exclude the cluster if a safe summary is impossible.

QUALITY FLOOR

If a user requests five stories and only three qualify, Bulletin sends three. If none qualify, it sends a short 'No meaningful updates matched your preferences' message. There are no filler stories.

07 / SYSTEM CHAPTER

Gemini as a controlled processing engine

Gemini is not a free-form chatbot inside the pipeline. It performs narrow, versioned tasks with strict schemas, explicit evidence references and fail-closed outcomes.

Provider abstraction

All AI access sits behind a provider interface so another provider or model can replace Gemini later. Prompts, request metadata and outputs are versioned independently from application code.

Separate AI tasks

Task	Responsibility	Required result
Classification	Category, geography, entities, central topics and sensitive flags	Strict structured JSON
Cluster verification	Decide whether candidate reports describe the same event	Accept, reject, conflict or insufficient evidence
Summarization	Create canonical headline, 3 to 4 sentence summary and why-it-matters line	Source-linked factual JSON
Localization	Create Hindi or Malayalam while preserving all canonical facts	Language version tied to canonical summary
Final verification	Compare proposed output with supplied evidence	Pass or explicit failure reasons

No conversational blocking

Gemini is never allowed to ask follow-up questions. Every task returns one machine-readable status: success, insufficient_evidence, conflicting_evidence, invalid_input or failed. Anything outside the schema is invalid output.

Prompt constraints

- Reference only supplied source IDs.
- Forbid unsupported facts, invented quotations and invented numbers.
- Preserve uncertainty and attribution.
- Separate facts from inference.
- Avoid sensational or clickbait headlines.
- Return insufficient evidence instead of improvising.
- Include prompt version, model and task metadata.

Retry policy

Malformed or unsupported output receives one tightly controlled correction attempt. If validation fails again, the task is marked failed, the cluster is excluded and the failure becomes visible to the admin. Endless retries are forbidden.

Final verification

Every generated summary receives a short independent verification call before it can be delivered. Verification runs once per shared summary, not once per subscriber. It returns pass or specific failure reasons.

FREE-TIER GUARD

The system tracks its own Gemini requests and token estimates, reserves capacity for verification, and stops new AI work before a conservative quota ceiling. Verification is never skipped to squeeze in more stories.

Shared summaries, languages and meaningful updates

The canonical story is a reusable product asset. It is generated once, verified once and localized only when needed.

Canonical English summary

Each eligible cluster receives one verified English summary containing category, rewritten headline, concise factual summary, why-it-matters line, source IDs, uncertainty markers and update status.

Lazy localization

Hindi and Malayalam are generated only when a subscriber first needs that cluster in the language. The version is stored and reused for later subscribers. Localization must preserve names, numbers, uncertainty, attribution and source references exactly.

Repeat-story suppression

- A cluster already delivered to a subscriber is excluded from later briefings.
- A rewritten headline or an additional publisher repeating the same facts is not an update.
- A meaningful new development creates an update version and may re-enter ranking.
- The email shows an Update label when the story legitimately returns.

Time windows

Frequency	Relevant window
Daily	Since the previous briefing, normally about 24 hours
Weekdays	Since the previous weekday delivery; Monday includes the weekend gap
Weekends	Since the previous weekend delivery
Weekly	Since the previous weekly delivery, up to roughly seven days

ORIGINAL ARCHITECTURE RETAINED

All valid news is ingested, normalized and clustered. Every eligible cluster is summarized and stored before personalization. A proposed demand-driven summarization change was rejected because it would add preference dependencies and delivery-time complexity.

09 / SYSTEM CHAPTER

Personalization without a black box

Personalization uses transparent structured metadata and deterministic scoring. It does not require a separate Gemini conversation for each user.

STEP 01 Load eligible clusters Use the subscriber's delivery window and verified summary state.	STEP 02 Remove repeats Exclude previously delivered cluster versions unless marked as meaningful updates.	STEP 03 Apply boundaries Require selected categories or an explicit custom-topic match.
STEP 04 Apply exclusions Block only when an excluded topic is central, not incidental.	STEP 05 Score relevance Combine topic, geography, recency, evidence and source quality.	STEP 06 Re-rank diversity Prevent one category or one subject from dominating.
STEP 07 Select quality set Return up to the requested count without filling weak positions.		

Preference boundaries

- A nationally important story outside selected categories is not inserted automatically.
- Custom topics are explicit interests and may qualify independently of categories.
- Excluded topics operate semantically: central involvement blocks; incidental mention does not.

Geographic order

National importance is ranked first, then state relevance, then city relevance. State and city signals personalize the result without allowing weak local material to displace stronger national reporting.

Diversity rule

- For 3 to 4 stories, normally no more than 2 come from one category.
- For 5 to 10 stories, normally no more than 40 percent come from one category.
- The limit relaxes when the user chose only one category or strong alternatives do not exist.
- Several stories about essentially the same subject are suppressed even if they sit in different categories.

Conceptual scoring model

Signal	Effect
Direct custom-topic match	Strong positive signal because the user explicitly requested it
Selected category match	Required eligibility unless a custom topic independently qualifies
National / state / city relevance	Descending geographic priority
Recency and meaningful development	Newer meaningful facts outrank repetitive updates
Multi-source and primary evidence	Raises confidence and ranking
Source quality	Rewards reliable reporting without automatically rejecting niche sources
Already delivered	Hard exclusion unless a meaningful update version exists
Central excluded topic	Hard exclusion

NO CLICK TRACKING

The MVP does not track source-link clicks. Personalization comes from explicit user choices and transparent rules, not hidden behavioral profiles or machine-learning inference.

10 / SYSTEM CHAPTER

Scheduling, idempotency and worker coordination

Minute-level scheduling is achieved with Supabase Cron calling protected Vercel functions. PostgreSQL coordinates claims and prevents duplicate work.

Subscriber schedule

Each subscriber stores next_delivery_at in UTC. The scheduler interprets the user's exact local time and IANA timezone, then calculates the next UTC instant for daily, weekday, weekend or weekly frequency.

Due delivery transaction

STEP 01
Find due subscriber

Query confirmed, active subscribers whose UTC delivery time has arrived.

STEP 02
Claim atomically

Lock or lease the schedule so another worker cannot claim the same slot.

STEP 03
Create delivery

Insert a delivery row with a unique subscriber plus scheduled-time key.

STEP 04
Advance schedule

Calculate and store the next future delivery time in the same transaction.

STEP 05
Queue delivery

Leave a resumable pending job for the email worker.

Idempotency

The unique idempotency key is conceptually subscriber UUID plus scheduled UTC delivery time. Even if cron overlaps, a function restarts or a network response is lost, the database cannot create a second scheduled delivery for the same slot.

Worker groups

Worker	Wake-up	Responsibility
RSS ingestion	About every 5 minutes	Claim due feeds, fetch, normalize and update health
Article processing	Every minute	Embed, cluster, verify, summarize and persist statuses
Briefing scheduler	Every minute	Create idempotent delivery records and advance schedules
Email delivery	Every minute	Rank stories, localize if needed, render, send and record
Cleanup	Daily	Expire technical data and unverified sign-ups according to policy

Pause and resume

- Pause immediately prevents pending unsent delivery.
- Resume calculates the next normal scheduled delivery.
- No catch-up email is sent merely because a user resumed.

DELIVERY TARGET

Under normal conditions, sending should begin within 1 to 2 minutes of the chosen time. The acceptable launch target is within 5 minutes. Final inbox arrival remains controlled by the recipient's email provider.

11 / SYSTEM CHAPTER

Email assembly and delivery

The last mile combines the selected stories, language and theme into email-safe HTML, then sends through one dedicated Gmail account.

Briefing composition

- Bulletin masthead and localized date.
- Personal greeting and actual delivered-story count.
- For each story: category, rewritten headline, 3 to 4 sentence summary, why it matters, sources and optional Update label.
- Footer: Exclusively prepared for the current subscriber name.
- Secure Manage briefing link in every briefing.

Themes

Light Editorial	Dark Intelligence
Pale ivory, charcoal typography, muted blue, fine dividers, bold Bulletin masthead and editorial rhythm. Default theme.	Near-black background, warm white type, restrained blue highlights and premium intelligence-report character without cyberpunk styling.

Subject and privacy

- Default subject: Your Bulletin - 12 July 2026, localized by briefing language.
- No clickbait story headlines in the subject.
- Source links go directly to publishers.
- No click tracking or personal tracking identifiers.

SMTP configuration

- Dedicated Gmail sender with two-factor authentication and an App Password.
- Nodemailer over SMTP port 465 or 587.
- Credentials stored only as server-side environment variables.
- The normal Gmail password is never stored or used by the application.

Retry behavior

Temporary SMTP failures retry after approximately 5, 15 and 60 minutes. Permanent failures do not retry indefinitely. Successful deliveries cannot be resent from the admin dashboard, protecting users from accidental duplicates.

EMAIL-CLIENT QA

Before launch, both themes must be tested in Gmail desktop and mobile, Apple Mail, Outlook where practical, and client light/dark modes. Implementation uses tables and inline styles suitable for email clients.

12 / SYSTEM CHAPTER

Subscriber identity and secure management access

Bulletin avoids passwords without allowing an email address alone to reveal preferences.

Database identity

- Each subscriber has a generated UUID primary key.
- Email is unique but is not the primary key.
- Duplicate subscriber records are blocked at the database level.

Signed management link

The permanent email link represents a public subscriber reference, token version and HMAC signature created with a server-only secret. The raw private token does not need to be stored in the database.

STEP 01

Receive link

The server receives the signed reference from an email.

STEP 02

Verify signature

Use timing-safe comparison and the server-only signing secret.

STEP 03

Check version

Reject links whose subscriber token_version has been incremented.

STEP 04

Issue session

Create a short-lived Secure, HttpOnly, SameSite cookie.

STEP 05

Clean redirect

Remove the private signature from the visible browser URL.

Existing email behavior

- Verified subscriber: send a secure management link; never show preferences based on typed email.
- Unverified sign-up: preserve pending preferences and send a fresh confirmation link.
- Unknown Manage briefing email: show New here? Create your Bulletin and prefill onboarding.
- Unverified records expire after seven days.

Verification safety

- Verification links expire after 24 hours and work only once.
- Requesting a new link invalidates older links.
- A GET request opens a confirmation page; a deliberate button performs confirmation.
- This prevents automated email scanners from activating a briefing accidentally.

Deletion safety

Unsubscribe opens an explicit confirmation page. Only a deliberate confirmation deletes the subscriber profile, preferences, schedule, access data and personal delivery data. A single email-link GET request never deletes anything.

KNOWN TRADEOFF

Anyone who obtains the actual private management link may access preferences. HTTPS, signed links, token invalidation, clean redirects and secure session cookies reduce risk, but a compromised inbox or forwarded email remains a real limitation of passwordless access.

13 / SYSTEM CHAPTER

Database model and retention

Supabase PostgreSQL is the source of truth for users, sources, articles, clusters, summaries and every delivery attempt.

Core tables

Table	Purpose
subscribers	Identity, confirmation, status, preferences, schedule, timezone, next delivery and theme
preference_versions	Thirty-day snapshots used for recovery and change diagnosis
sources	Feed configuration, language, geography, reliability, schedule and health
articles	Normalized public RSS article records and processing state
story_clusters	Canonical grouped events, evidence state, update lineage and embeddings
story_cluster_articles	Many-to-many evidence links between clusters and source articles
cluster_summaries	Canonical and localized versions, prompt/model metadata and verification state
deliveries	One record per scheduled briefing attempt with unique idempotency key
delivery_stories	Exact cluster versions included in each briefing
admin_audit_log	Owner actions, targets, timestamps and outcomes

Retention

Data	Initial retention
Raw RSS metadata	14 days
Story clusters and summaries	30 days
Delivery records	90 days
Preference versions	30 days
Unverified sign-ups	7 days
Confirmed subscriber personal data	Until confirmed unsubscribe/deletion

Atomic preference updates

Managed preferences use one explicit Save changes action. The complete input is validated, the previous version is snapshotted and the new state is committed as one transaction. Theme selection is the only preference that saves immediately.

Row Level Security

Public clients receive no direct broad access to subscriber data. Server-only functions use narrowly scoped service credentials, and database policies protect tables from accidental frontend exposure.

14 / SYSTEM CHAPTER

Private owner operations

The admin dashboard provides broad operational control while deliberately withholding dangerous shortcuts that could send duplicates or silently change user intent.

Visibility

- No public navigation or footer link.
- Excluded from sitemap and search indexing.
- Hidden route protected by allowlisted owner email and short-lived one-time login link.

Owner controls

- Enable, disable, test and manually refresh RSS sources.
- Quarantine a story or cluster so it cannot be delivered.
- Reprocess failed clusters and regenerate failed summaries within quota safeguards.
- Emergency pause and resume for all email delivery.
- Pause and resume Gemini processing when quota becomes unsafe.
- Search subscriber status, schedule and recent delivery results.
- Resend verification or management emails.
- Pause/resume a subscriber, invalidate old management links or cancel pending unsent delivery.
- Delete a subscriber only after strong confirmation.
- Review an audit log of every admin action.

Retry boundaries

- Retry temporary RSS, processing, Gemini and SMTP failures.
- Never retry completed jobs.
- Never retry permanent email failures indefinitely.
- Never manually resend a successful briefing.

What the admin cannot do silently

The owner cannot impersonate a subscriber, silently edit personal preferences or bypass idempotency. Full control means safe operational control, not a button that can violate trust.

Alerts

Repeated delivery failures, worker stalls, quota exhaustion and widespread source failure trigger urgent owner email. Isolated or low-priority warnings stay in the dashboard to avoid alert fatigue.

15 / SYSTEM CHAPTER

Failure states and recovery

Every important failure must become explicit, resumable and visible. Silent loss is treated as a product defect.

Failure	System response	User impact
RSS timeout	Backoff, isolate source, preserve other fetches	None unless source remains unavailable
Malformed feed	Mark parser failure, alert after threshold	Other sources continue
Worker crash	Lease expires; another worker resumes	Short delay, no duplicate job

Failure	System response	User impact
Gemini malformed JSON	One repair attempt, then failed state	Story excluded
Gemini quota guard	Stop new AI tasks before ceiling	Use already verified stories or empty briefing
Cluster conflict	Attribute disagreement or quarantine	Potentially fewer stories
SMTP temporary failure	Retry at about 5, 15 and 60 minutes	Delayed email
SMTP permanent failure	Stop retries and expose admin status	No repeated unwanted sends
Duplicate scheduler call	Unique key rejects second delivery	No duplicate briefing
Database update conflict	Transaction rolls back; previous version remains	User can safely retry

Fail closed, not open

When evidence, AI output, quota or delivery state is uncertain, Bulletin chooses fewer stories, a delayed operation or a visible failure. It never invents content, silently overwrites preferences or risks a duplicate email to preserve the appearance of success.

16 / SYSTEM CHAPTER

Encrypted backup and disaster recovery

A database is not protected merely because it is hosted. Bulletin requires a separate, encrypted and restore-tested copy.

Backup design

- Automated daily Supabase database export.
- Encrypt before the backup leaves the process.
- Store in a dedicated Google Drive folder at zero expected storage cost for MVP scale.
- Keep seven daily backups and four weekly backups.
- Store the encryption key separately from Drive and deployment source code.
- Run a real restore test monthly during the private beta.

Why restore testing matters

A file called backup is not evidence of recoverability. The restore test verifies credentials, encryption keys, database compatibility, extensions such as pgvector, constraints, functions and application behavior against the recovered data.

Recovery sequence

STEP 01 Declare incident Stop writes and email delivery to prevent further damage.	STEP 02 Select backup Choose the newest known-good encrypted copy.	STEP 03 Decrypt safely Use the separately stored recovery key in a controlled environment.
--	--	--

STEP 04**Restore database**

Recreate schema, data, extensions, functions and constraints.

STEP 05**Validate**

Check subscriber counts, schedules, deliveries and cluster integrity.

STEP 06**Resume cautiously**

Recalculate due work without duplicating successful deliveries.

17 / SYSTEM CHAPTER

Reliability and launch gates

Bulletin is not ready because it works once. It is ready only after failure has been deliberately introduced and the system still protects user trust.

Required simulations

- One hundred subscribers becoming due together.
- Thousands of preference updates with zero corruption.
- Duplicate scheduler calls and overlapping worker invocations.
- Worker crashes at each major state transition.
- Temporary and permanent Gmail failures.
- Malformed Gemini JSON, unsupported facts and quota exhaustion.
- RSS timeouts, invalid XML, oversized payloads and disabled sources.
- Daily, weekday, weekend and weekly delivery schedules.
- Multiple timezones and daylight-saving transitions.
- Backup creation and full restoration.

Launch gate

- Zero corrupted or lost preferences.
- Zero duplicate scheduled deliveries.
- Every processing failure visible to the admin.
- At least 99 percent of test sends started within five minutes of selected time.
- A 7 to 14 day soak test completed successfully.

Rollout

Stage	Audience	Purpose
1	Owner-controlled accounts	Prove end-to-end correctness and inspect every email
2	Five trusted users	Observe real schedules, devices and preference changes
3	Twenty users	Validate worker capacity, quota and source relevance
4	Fifty to one hundred users	Private beta operating target

NO HEROIC LAUNCH

If the free architecture cannot meet the launch gate, Bulletin does not launch publicly. The same worker logic can move to a paid always-on host later without changing the product model.

18 / SYSTEM CHAPTER

Implementation map

The architecture is built in dependency order so security and data integrity arrive before the intelligence pipeline and visual polish.

Phase	Outcome
1. Foundation	Next.js, TypeScript, validation, configuration, logging and centralized product name
2. Database	Schema, pgvector, constraints, RLS, job-claim functions and migrations
3. Onboarding	Five-step validated flow, draft protection, review and consent
4. Secure access	Verification, signed links, token invalidation, sessions and management
5. Landing page	Approved Light Editorial design with focused responsive fixes
6. Ingestion	Source catalogue, scheduling, parsing, normalization and health
7. Clustering	Embeddings, candidate search, consistency checks and threshold tests
8. Gemini	Provider abstraction, schemas, prompts, retries, verification and quotas
9. Personalization	Eligibility, scoring, diversity and repeat suppression
10. Scheduling	Timezone-safe next delivery and idempotent creation
11. Email	Themes, localization, SMTP, retries and client testing
12. Admin	Health, alerts, safe controls and audit log
13. Backups	Encrypted automated Drive backup and restore drills
14. Reliability	Failure simulation, soak test and staged rollout

Confirmed stack

- Next.js, TypeScript and Tailwind CSS
- GSAP only where selected landing-page motion adds value
- Supabase PostgreSQL, pgvector and Supabase Cron
- Protected Vercel Functions for stateless workers
- Gemini behind a provider abstraction
- Nodemailer and Gmail SMTP
- Shared TypeScript types and schema validation such as Zod
- One monorepo

IMPORTANT BOUNDARY

This blueprint is complete planning, not authorization to begin building. Development starts only after explicit owner approval of the consolidated implementation plan.

The interesting logic - what makes Bulletin special

The defensible value is not a single AI prompt. It is the disciplined composition of many understandable rules.

1. Shared intelligence, personal delivery

Public news is processed once; only selection and delivery vary by person. This creates both efficiency and consistency.

2. Similarity proposes; consistency decides

Embeddings discover possible event matches, but named entities, geography, time, numbers and event nature prevent unsafe merging.

3. Every source survives summarization

The canonical story does not erase the reporting. Readers can inspect every attached publisher, and the system can re-evaluate evidence later.

4. Personalization is explicit

Categories, custom topics, exclusions and geography are visible choices. There is no hidden click-tracking profile in the MVP.

5. Exclusions understand context

A blocked term removes a story only when it is central, preventing naive keyword filters from discarding useful reporting.

6. Updates require actual change

A new headline or additional publisher does not revive an old story. Only a meaningful development creates an Update.

7. Reliability beats completeness

Three strong stories are better than five padded ones. An empty briefing is better than unsupported text.

8. Every scheduled delivery has one identity

The subscriber plus scheduled UTC time becomes a database-enforced idempotency key, making duplicate delivery structurally difficult.

9. AI is boxed into narrow jobs

Classification, cluster verification, summarization, localization and validation have separate schemas, statuses and failure handling.

10. Owner control remains safe

The admin can pause, quarantine, retry and recover without bypassing the rules that protect subscribers.

20 / SYSTEM CHAPTER

Plain-language glossary

A compact reference for the technical ideas used throughout the blueprint.

Term	Meaning
Atomic operation	Several database changes succeed together or none of them do.
Canonical story	The shared representation of one real-world news event.
Cluster	A group of articles that report the same event.
Conditional HTTP request	A request that downloads a feed only when it changed.
Embedding	A numeric representation of meaning used for semantic comparison.
Fail closed	Reject or pause uncertain work instead of allowing unsafe output.
HMAC signature	A server-created cryptographic proof that a private link is genuine.
HttpOnly cookie	A browser session cookie unavailable to frontend JavaScript.
Idempotency	Repeating the same request produces no duplicate side effect.
Lease	A temporary database claim allowing crashed work to be recovered.
Localization	Producing a language version while preserving the same facts.
pgvector	A PostgreSQL extension for storing and searching embeddings.
RLS	Row Level Security policies controlling which database rows may be accessed.
Semantic exclusion	Blocking a topic only when it is central, not incidentally mentioned.
Soak test	Running the complete system continuously to reveal delayed failures.
Stateless function	A worker that keeps durable state in the database, not in memory.
Token version	A subscriber counter used to invalidate all older management links.

A calm product. A serious system.

Bulletin appears simple to the reader because the complexity is handled behind the scenes with restraint.

The complete chain

Verified sources are fetched once. Cheap duplicates disappear first. Semantic candidates are tested against real event facts. Evidence remains attached. Gemini performs small structured tasks, then a final verifier checks the shared summary. The database stores every important state. Transparent rules personalize without hidden tracking. UTC scheduling and idempotency protect delivery. Signed links protect preferences. The owner can intervene safely. Backups and launch gates protect the product itself.

The reader receives only what matters: a concise, trustworthy briefing at the chosen time.

Built through patience, careful questions and a refusal to trade trust for speed.

BULLETIN

Stay informed without surrendering your time and attention.